

anggota:

-Kanda Rifki Alfaz

-Danang Adiwijaya

-M Khalid Al Rejeki

LAPORAN PERANCANGAN SISTEM AUTHSHIELD

A. Latar Belakang

Dalam era transformasi digital saat ini, keamanan data pengguna telah menjadi prinsip utama dalam pengembangan aplikasi berbasis web. Di antara berbagai jenis data sensitif, data autentikasi seperti *password* merupakan target utama bagi para peretas karena berfungsi untuk mengakses hak istimewa pengguna. Kegagalan dalam melindungi integritas data *password* seringkali berakar pada penggunaan algoritma *hashing* tradisional yang cepat atau penyimpanan data dalam bentuk *plaintext* [1].

Kerentanan sistem autentikasi ini sangat dipengaruhi oleh tingkat kekuatan kata sandi yang dibuat oleh pengguna, yang dalam ilmu kriptografi diukur melalui tingkat entropi [2]. Entropi adalah ukuran seberapa acak, tidak terduga, dan bervariasinya sebuah kata sandi berdasarkan panjang dan ragam karakter yang digunakan. Kata sandi dengan entropi yang tinggi akan memperluas ruang pencarian kombinasi sehingga membutuhkan waktu yang lama bagi peretas untuk menebaknya.

Masalah entropi rendah ini diperparah oleh penggunaan algoritma *hashing* lama seperti MD5 atau SHA-1 yang awalnya dirancang hanya untuk memeriksa integritas data dengan performa super cepat. Di era komputasi modern yang didukung oleh *Graphics Processing Unit* berperforma tinggi, kecepatan ini justru menjadi celah keamanan yang fatal karena penyerang dapat menguji miliaran kombinasi hash per detik. Akibatnya, kata sandi pengguna yang berentropi rendah dapat dipatahkan hanya dalam hitungan detik melalui serangan *brute-force* atau pencocokan dengan basis data *rainbow table* [3].

Dalam menghadapi tantangan tersebut, National Institute of Standards and Technology (NIST) menegaskan bahwa mekanisme autentikasi modern harus menggunakan fungsi yang sengaja dirancang lambat (*computationally expensive*) [4]. Salah satu skema kriptografi yang memenuhi standar ini adalah algoritma Bcrypt [5]. Keunggulan utama Bcrypt terletak pada parameter *cost factor* yang memungkinkan pengembang untuk memperlambat waktu pemrosesan *hashing* secara sengaja. Efek dari *cost factor* secara langsung mengatasi kelemahan kata sandi berentropi rendah dengan

memaksa komputer penyerang bekerja berkali-kali lipat lebih keras dan lama untuk menguji satu kombinasi kata sandi. Meskipun demikian, ketergantungan penuh pada data yang tersimpan di dalam basis data masih menyisakan *single point of failure* jika peretas berhasil menembus basis data melalui celah *SQL Injection* dan melakukan serangan secara *offline*.

Untuk mengurangi risiko kebocoran basis data tersebut, diperlukan sebuah lapisan pertahanan berlapis melalui proyek perancangan sistem registrasi dan login yang dinamakan AuthShield. AuthShield dikembangkan untuk melampaui implementasi standar Bcrypt dengan menerapkan gabungan penggunaan *Salt* unik dengan *Secret Key* yang dibatasi pada level *environment variable* web backend. Penambahan *Secret Key* bertindak sebagai penyedia entropi eksternal yang kuat pada kata sandi sebelum diproses oleh Bcrypt. Melalui arsitektur AuthShield ini, jika terjadi situasi buruk di mana basis data bocor, peretas tetap tidak akan bisa melakukan rekonstruksi atau *brute-force* karena tidak memiliki komponen *Secret Key* yang disimpan terpisah di luar basis data tersebut.

B. Tujuan

Adapun tujuan dari perancangan dan pembuatan sistem autentikasi AuthShield ini adalah:

1. Menerapkan sistem validasi yang mampu memastikan kekuatan kata sandi pengguna menggunakan pendekatan teori entropy Shannon.
2. Mengimplementasikan algoritma kriptografi bcrypt dengan menggabungkan *Salt*, *Pepper* dan *Secret Key* dalam mengamankan dari ancaman kebocoran data.
3. Merancang alur registrasi dan login yang tahan terhadap serangan berbasis *Rainbow Table* maupun *offline brute-force attack*.

C. Tinjauan Pustaka

Landasan utama dalam membangun sistem keamanan autentikasi berbasis web berakar pada penerapan fungsi hash kriptografi searah. Secara fundamental, fungsi hash kriptografi adalah sebuah algoritma matematis yang menerima masukan data dengan panjang variabel dan mengubahnya menjadi representasi string berukuran tetap yang unik. Dalam konteks perlindungan kredensial, proses ini harus bersifat satu arah yang berarti nilai hash yang dihasilkan tidak dapat didekripsi atau dikembalikan lagi ke bentuk *plaintext* aslinya [2]. Oleh karena itu, validasi identitas pengguna tidak dilakukan dengan cara membongkar kembali kata sandi yang tersimpan melainkan dengan merekonstruksi ulang proses *hashing* pada kata sandi yang dimasukkan saat login lalu mencocokkan hasilnya dengan nilai hash yang telah berada di dalam basis data.

Kekuatan awal dari sebuah kata sandi sebelum memasuki proses *hashing* ditentukan oleh ukuran matematis dari tingkat keacakan dan prediktabilitasnya yang dikenal sebagai entropi kata sandi. Evaluasi nilai entropi merupakan langkah preventif yang krusial untuk mengukur seberapa tahan sebuah kata sandi terhadap serangan tebakan acak. Kalkulasi tingkat keacakan ini dihitung menggunakan rumus Shannon untuk entropi informasi, yaitu:

$$E = L \times \log_2(R)$$

Dalam formula tersebut, variabel E merepresentasikan nilai entropi dalam satuan bit, sedangkan L menunjukkan panjang atau jumlah karakter dari kata sandi, dan R merupakan ukuran *pool* karakter atau ragam jenis karakter yang digunakan, seperti kombinasi huruf kecil, huruf besar, angka, dan simbol [2]. Semakin tinggi nilai entropi (E), maka ruang pencarian kombinasi yang harus dipecahkan oleh penyerang akan meluas secara eksponensial sebesar 2^E . Sebaliknya, jika pengguna menciptakan kata sandi dengan pola yang mudah ditebak, nilai entropinya akan sangat rendah sehingga sistem memerlukan mekanisme pertahanan tambahan di sisi *backend* untuk memperlambat laju komputasi peretas.

Untuk menambal kelemahan kata sandi yang berentropi rendah tersebut, standar keamanan modern beralih menggunakan fungsi turunan kunci yang bersifat adaptif, salah satunya adalah algoritma Bcrypt yang dikembangkan berdasarkan *cipher* Blowfish [5]. Keunggulan utama Bcrypt dibandingkan algoritma lama seperti MD5 atau SHA-256 terletak pada sifatnya yang secara inheren sengaja dirancang berjalan lambat. Parameter kunci yang mengatur perlambatan komputasi ini disebut sebagai *cost factor* atau *work factor*. Hubungan antara *cost factor* dengan jumlah iterasi internal yang dilakukan oleh Bcrypt dirumuskan secara eksponensial sebagai berikut:

$$\text{Iterasi} = 2^{\text{cost}}$$

Melalui rumus tersebut, setiap kenaikan satu digit nilai *cost factor* akan melipatgandakan jumlah kerja komputasi dan waktu yang diperlukan untuk menghasilkan satu nilai hash. Karakteristik adaptif ini memungkinkan sistem untuk mengimbangi peningkatan performa perangkat keras komputer di masa depan; pengembang cukup menaikkan nilai parameter *cost* secara berkala untuk memastikan

bahwa upaya serangan *brute-force* secara massal tetap membutuhkan biaya komputasi dan waktu yang tidak masuk akal bagi peretas [5].

Kendati Bcrypt sangat tangguh dalam memperlambat serangan, algoritma ini tetap membutuhkan komponen pengaman tambahan berupa *Salt* dan *Secret Key* untuk mencapai tingkat keamanan yang optimal. *Salt* adalah string acak unik yang digenerasikan secara otomatis untuk setiap pengguna dan berfungsi untuk memastikan bahwa dua kata sandi yang sama akan menghasilkan nilai hash yang sepenuhnya berbeda, sehingga secara efektif menggagalkan serangan berbasis *rainbow table* atau kamus pra-komputasi [3].

Namun, karena *Salt* wajib disimpan di dalam basis data bersama dengan nilai hash agar dapat digunakan kembali pada proses verifikasi, ketergantungan ini memunculkan risiko tunggal jika basis data tersebut bocor. Guna memitigasi celah tersebut, penggunaan *Secret Key* menjadi proteksi lapis kedua yang sangat direkomendasikan [4]. Berbeda dengan *Salt*, *Secret Key* tidak ikut disimpan di dalam basis data melainkan diisolasi di luar sistem, biasanya pada level konfigurasi server atau *environment variable* aplikasi. Dengan menggabungkan seluruh komponen tersebut ke dalam formula kriptografi sebelum diproses oleh Bcrypt, kredensial pengguna akan tetap terlindungi secara kokoh meskipun peretas berhasil mengunduh seluruh isi basis data, karena peretas tidak memiliki *Secret Key* eksternal yang diperlukan untuk merekonstruksi hash secara luring.

D. Perancangan

Berdasarkan rencana pengembangan AuthShield, perancangan sistem ini mencakup dua alur kerja utama:

1. Alur Kerja Registrasi (*Registration Workflow*)

Fokus pada alur ini adalah proses validasi, pengamanan melalui kriptografi, dan penyimpanan yang persisten.

a. Input Data

Pengguna mengirimkan *Username/Email*, *Password*, dan *Konfirmasi Password*.

b. Validasi Awal

Sistem mencocokkan kata sandi dengan kolom konfirmasi. Sistem kemudian menghitung kekuatan kata sandi menggunakan rumus entropi. Jika tidak memenuhi standar keamanan, sistem menolak *request* dan memberikan pesan *error*; jika valid, alur dilanjutkan.

c. Kriptografi & Penggabungan

Sistem menghasilkan *string Salt* secara acak untuk pengguna baru. Sistem merangkai kata sandi pengguna bersama *Salt* dan *Secret Key* dari server. Format perangkaiannya adalah: Password + Salt + Secret Key.

d. Hashing

Rangkaian tersebut dimasukkan ke fungsi *hash* (Bcrypt).

e. Penyimpanan

Sistem menyimpan data *Username/Email*, *Hash Password*, dan *Salt* ke dalam basis data. *Secret Key* tidak dimasukkan ke basis data, melainkan dipertahankan di *environment* aplikasi.

2. Alur Kerja Login (*Login Workflow*)

Alur ini dirancang untuk merekonstruksi formula dari tahap registrasi tanpa harus mendekripsi data.

- a. Input Data & Pencarian: Pengguna memasukkan *Username/Email* dan *Password* login. Sistem menelusuri eksistensi pengguna di basis data. Demi keamanan, jika pengguna tidak ditemukan, aplikasi memunculkan pesan *error* generik.
- b. Pengambilan Data Target: Jika pengguna ada, sistem memuat *Hash Password* lama dan *Salt* milik pengguna dari basis data.
- c. Rekonstruksi & Hashing Ulang: Sistem menyiapkan kata sandi login, *Salt* basis data, dan *Secret Key* server. Elemen-elemen ini digabungkan mengikuti format registrasi (Password Login + Salt Database + Secret Key) dan kemudian di-*hash* ulang dengan algoritma Bcrypt.
- d. Verifikasi: Sistem membandingkan hasil *hash* baru ini dengan *Hash Password* yang ditarik dari basis data.
 - Jika Cocok: Kata sandi valid. Sistem merilis sesi (*Session*) atau token JWT lalu mengarahkan akses ke halaman *Dashboard*.
 - Jika Berbeda: Autentikasi gagal dan sistem menampilkan *error* penolakan akses.

E. Pengujian

Pada tahapan perkembangan sistem AuthShield saat ini, fokus utama diarahkan pada eksperimen awal untuk menguji karakteristik performa dan menentukan nilai *cost factor* yang paling ideal pada algoritma Bcrypt. Adapun code yang digunakan dalam pengujian sebagai berikut:

```

const bcrypt = require('bcrypt');
const { performance } = require('perf_hooks');

const password = "passwordSuperRahasia123";
const SECRET_KEY = "KunciRahasiaSistem123";
const FIXED_SALT = "$2b$10$abcdefghijklmnopqrstuvwxyz";

async function testBcryptSpeed() {
  for (let cost = 10; cost <= 14; cost++) {

    const start = performance.now();

    const passwordWithPepper = password + SECRET_KEY;

    const salt = await bcrypt.genSalt(cost);

    const hash = await bcrypt.hash(passwordWithPepper, salt);

    const end = performance.now();
    const duration = (end - start).toFixed(2);

    console.log(Cost Factor: ${cost});
    console.log(Salt          : ${salt});
    console.log(Hash          : ${hash});
    console.log(Waktu         : ${duration} ms);
    console.log("-----");
  }
}

testBcryptSpeed();

```

Skenario eksperimen ini dirancang menggunakan skrip berbasis Node.js dengan mengombinasikan kata sandi uji coba "passwordSuperRahasia123" dan *Secret Key* eksternal "KunciRahasiaSistem123". Melalui pemanfaatan pustaka bcrypt dan modul penanda waktu performance.now(), pengujian dilakukan secara berulang untuk mengukur durasi pemrosesan komputasi *hashing* searah pada skala rentang *cost factor* dari tingkatan 7 hingga 14.

```

C:\HackLangit\Cripto>node bekrip.js
Cost Factor: 7
Salt      : $2b$07$/8S9WkXx8ENqp298bxZ270
Hash      : $2b$07$/8S9WkXx8ENqp298bxZ270XxE6p1qIO9eGQ.2pPQs9sIFIXR8.f0m
Waktu     : 7.56 ms
-----
Cost Factor: 8
Salt      : $2b$08$uIGRm7CT0bGKQX.NTzRQ20
Hash      : $2b$08$uIGRm7CT0bGKQX.NTzRQ20juwENLOYqr0TOWuyCR3Kr93gn5E4ETu
Waktu     : 13.36 ms
-----
Cost Factor: 9
Salt      : $2b$09$ZkjCTqHGy.xcPNyE5a47Be
Hash      : $2b$09$ZkjCTqHGy.xcPNyE5a47BeRe2chewS7XSwu7i4G.OBWGTqeeqglce
Waktu     : 27.06 ms
-----
Cost Factor: 10
Salt      : $2b$10$8BH3kwoRX0dkxF8d9iLjve
Hash      : $2b$10$8BH3kwoRX0dkxF8d9iLjve/D4LtC0NkRhC5IRbrwUbX26jNHvX/qe
Waktu     : 52.75 ms
-----
Cost Factor: 11
Salt      : $2b$11$sgafpi.bccSBenDIyYRXCu
Hash      : $2b$11$sgafpi.bccSBenDIyYRXCuZwj00D8MheSCWcwjD7luV3aA0knIZAK
Waktu     : 101.34 ms
-----
Cost Factor: 12
Salt      : $2b$12$ZPOccR1WvaHhmnVoqDjNe
Hash      : $2b$12$ZPOccR1WvaHhmnVoqDjNeIrxwuwN60kmMPL5xJlPMHbgR0IsaZ8y
Waktu     : 200.44 ms
-----
Cost Factor: 13
Salt      : $2b$13$Sot.b/qy58/PhoQzhM5Fo0
Hash      : $2b$13$Sot.b/qy58/PhoQzhM5Fo0NO34eHc7sIwcNCQf2oANaz4rj1Kw8YK
Waktu     : 398.59 ms
-----
Cost Factor: 14
Salt      : $2b$14$a92PHBMQkgCU0jy7oZ5bru
Hash      : $2b$14$a92PHBMQkgCU0jy7oZ5bruSr6J5EyEszE2mV2/Zbp1WBZMmyiK1ce
Waktu     : 829.77 ms
-----

```

Gambar 1 Hasil Percobaan *Cost Factor*

Hasil eksekusi skrip pada terminal lokal memberikan data empiris yang membuktikan sifat eksponensial dari arsitektur Bcrypt. Pada tingkat kesulitan terendah (*cost factor* 7), sistem hanya membutuhkan waktu pemrosesan sebesar 7,56 ms, namun durasi tersebut melonjak hingga dua kali lipat pada setiap kenaikan satu digit tingkatan kesulitan. Pola penggandaan waktu ini terlihat sangat jelas pada *cost factor* 11 (101,34 ms), *cost factor* 12 (200,44 ms), hingga memuncak pada *cost factor* 14 yang menghabiskan waktu hingga 829,77 ms. Selain itu, rekaman log terminal menunjukkan bahwa nilai *cost factor* secara transparan dilekatkan di awal string hash melalui notasi penanda seperti \$2b\$07\$ hingga \$2b\$14\$.

Laporan progres eksperimen ini menjadi landasan krusial bagi tim untuk menentukan konfigurasi terbaik sebelum mengimplementasikannya secara penuh pada aplikasi web AuthShield. Berdasarkan standar kenyamanan pengguna aplikasi web di mana toleransi waktu tunggu respon login berada di bawah 500 ms, hasil pengujian ini menyimpulkan bahwa penggunaan *cost factor* 12 atau 13 adalah pilihan yang paling optimal. Konfigurasi pada rentang tersebut dinilai berhasil mencapai titik keseimbangan karena mampu memberikan ketahanan ekstra terhadap potensi serangan *brute-force* luring tanpa mengorbankan performa kecepatan server backend.

DAFTAR PUSTAKA

- [1] P. C. v. Oorschot, *Computer Security and the Internet: Tools and Jewels*. Springer, 2012.
- [2] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.
- [3] J. Kelsey, B. Schneier, C. Hall, and D. Wagner, "Secure applications of low-entropy keys," in *Information Security: First International Workshop ISW'97*, Springer, 1998.
- [4] W. E. Burr, D. F. Dodson, and E. M. Newton, "Electronic Authentication Guideline," National Institute of Standards and Technology (NIST), Special Publication 800-63B, 2017.
- [5] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proceedings of the 1999 USENIX Annual Technical Conference (ATC)*, 1999.